Repair, refinement, and acceptance: a root-cause analysis

Status: applied as D79. The pre-D79 names in this note are historical; the implementation now uses `FixedSpecificationAcceptance` and `NearestSentence`. The analysis is retained because it explains why the typed proposal/advice split is the long-term model.

Executive summary

Scale currently uses the word **repair** for three different operations:

1. correcting a malformed Prose sentence;
2. refining a specification by appending a guard or refinement;
3. accepting an implementation patch against a frozen specification.

Those operations have different inputs, different outputs, different trust boundaries, and different lifetimes. The code partially reflects that distinction, but not completely:

- `RepairSearch` is token-level sentence correction.
- `Refiner` and `ScaleFile.append` are the real specification-refinement path.
- `Repair.accept` is the fixed-spec implementation gate.

The suspected earlier mistake is not "the gate forgot to call the search." It is that the model represents a specification edit as an optional string inside an implementation patch (`Patch.SpecEdit`), while a separate mode function (`Repair.appendInMode`) and a separate refiner already own specification changes. That creates an invalid state the gate must reject at runtime, duplicates the fixed-spec rule, and makes "repair" sound like one operation when it is three.

The A/B/C options are therefore surface choices:

- A (gate calls search) treats the symptom and is ill-typed without changing the gate's input.
- B (keep separate) is safe but leaves the overloaded model in place.
- C (edge advisor) is a useful UX adapter, but it is not a root-cause fix.

The architectural option is **D: separate typed proposals from advice**, with a linear frozen-spec capability. In fixed-spec mode a specification change should be **unrepresentable**, not merely rejected; diagnostics should carry structured **suggestions**, not a string called `Repair`; and the acceptance gate should consume only an implementation proposal.

Scale in one page, for a designer who does not know it

Scale is a concatenative, stack-oriented language and a verification harness. The details that matter here are independent of the stack:

- A **specification** is a sentence in the Prose sentence grammar, for example "Every player can jump." It is parsed into a sentence, then projected into a property and an SMT obligation.
- An **implementation** is a typed boolean expression over fields, for example `health > 1`.
- A **witness** is a concrete input plus field values. A **corpus** is a checked-in list of witnesses that must not regress.
- A **fixed-spec** run freezes the specification. The implementation may change; the specification may not. A **patch** that edits the specification is rejected before solver work.
- An **exploratory** run may refine the specification: a refiner proposes a guard or refinement and appends it to the `.scale` document.

The image is append-only. Execution state is separate and never serialized. This matters because a change is naturally a new image value, not a mutation.

What the code actually does

Artifact	Current representation	What it actually is
`RepairSearch`	`repair-search.fs`; `Candidate { Tokens; Text; Distance }`	Sentence advice: the nearest token sequence the grammar accepts
`Repair`	`repair.fs`; `accept`, `Patch`, `Verdict`	Fixed-spec implementation acceptance: a proof-backed decision
`Patch`	`Body` plus `SpecEdit: string option`	An implementation change with an optional specification-change trap
`Refiner`	`refiner.fs`; `RefinementResult`	Specification refinement: a proposed guard or widening of the spec
`Violation`	`violations.fs`; `Repair: string option`	Diagnostic advice: a human-readable hint, not a proposal
`Repair.Mode`	`Exploratory | FixedSpecification`	A run setting, not a type-level capability

Evidence for the mismatch

1. `Patch.SpecEdit` is an optional string. The comment says it exists so a generator "cannot smuggle a spec change through the code path"; carrying one is a rule violation. But in production the field is only ever constructed as `None` (`Repair.replaceBody`); the only `Some` construction in the tree is a test that checks the rejection. The field is a runtime trap, not a feature.
2. `Repair.accept` rejects `SpecEdit` before solver work (`repair.fs:264`). `Repair.appendInMode` separately rejects specification writes in `FixedSpecification` mode (`repair.fs:328`). The same invariant is enforced in two places.
3. `Refiner.refine` is the real specification-change path. The demo appends a refiner result with `ScaleFile.append` in `Exploratory` mode. That path is already typed enough to carry a `RefinementResult` with a guard and provenance.
4. The gate checks a `Target.SpecificationHash` at runtime. The specification is not a typed, content-addressed value in the patch; the "same spec" fact is a string comparison after the fact.
5. `RepairSearch` is token-level. It searches one-edit neighbours and defaults to the parser as its only acceptance check. It cannot know semantic equivalence, cannot inspect an implementation body, and is not used by the gate.
6. `Violation.Repair` is a `string option`. The display layer says "Repair: …". There is no structured `Suggestion` with a target, span, provenance, or applicability. `Presentation.withNearest` fills the string from `RepairSearch`; that is the only bridge between diagnostics and search.
7. `Repair.Mode` exists, but `Repair.accept` does not take a mode. The mode is used by `appendInMode` and by the demo. The gate's contract is therefore "fixed spec" regardless of the surrounding mode, while the spec-writer has a separate mode-sensitive rule.

The three meanings of "repair"

Operation	Input	Output	Trust boundary
Sentence correction	malformed tokens	nearest legal sentence	diagnostic advice; no proof
Specification refinement	spec + witness/guard	appended refinement	exploratory only; changes the contract

Operation	Input	Output	Trust boundary
Implementation acceptance	frozen spec + patch + corpus	Accepted/Rejected	proof-backed gate; never edits the spec

A single word for all three hides the fact that one changes the **contract** (the spec), one changes the **implementation**, and one changes only the **text the user is trying to write**. The first is an edit to the problem statement. The second is an edit to the solution. The third is a spelling correction to the problem statement, not yet a proposal.

Root-cause hypotheses

R1 — Overloaded vocabulary (high confidence). "Repair" names three different arrows: text → legal text, spec → stronger spec, implementation → accepted implementation. The names `Repair`, `RepairSearch`, and `Violation.Repair` are symptoms of one word doing three jobs.

R2 — `Patch.SpecEdit` is an invalid state made representable (high confidence). `Patch` is an implementation change with an optional specification change. That is a mixed sum type encoded as a record with an option. It violates the project's own "parse, don't validate" preference. A fixed-spec patch should not be able to carry a spec edit at all.

R3 — The frozen spec is a runtime hash, not a capability (medium-high confidence). A `FrozenSpecification` token, or a `SpecificationHash` in the patch type, would make the invariant structural. Today the gate recovers the invariant by comparing strings and rejecting an optional field.

R4 — Advice and decision are conflated (high confidence). A diagnostic suggestion and an acceptance verdict are different results. `Violation.Repair` is a string hint; `Repair.Verdict` is a decision. They share the word "repair" but not a type, an evidence model, or a lifetime.

R5 — Two languages, one repair abstraction (medium confidence). The specification is the Prose sentence grammar; the implementation is an expression language. Token edit distance is meaningful for the first and meaningless for the second. A single repair abstraction cannot cross that boundary cleanly.

R6 — The append-only image is not used for proposals (medium confidence). The image naturally represents a change as a new value. A proposal could be an image-resident value with a target, span, body, and evidence. Today the proposal is a host record plus a string field, which is why the fixed-spec invariant needs runtime checks.

R7 — Feature richness at the wrong layer (medium confidence). Wanting one user-facing "repair" flow is reasonable. Putting that flow into the acceptance gate is not. The project's principle should be "rich edge, simple core," not "one core function that branches on mode."

Why Option A is ill-typed

`Repair.accept` takes a `Patch` and a target. It has no malformed sentence to search. `RepairSearch` takes tokens and a vocabulary. It has no implementation body, no corpus, no proof, and no spec hash.

To make A work, one of these must happen:

- the gate gains a sentence input and calls search on it; or
- `Patch.SpecEdit` becomes a string the gate searches; or
- the result type grows an optional suggestion and the gate does both jobs.

All three change the gate's contract. The first is a second input language. The second is the invalid state R2. The third is Option C wearing the gate's name.

A is not the simple option. It is the option that hides a mode split inside a function that currently has a clean proof boundary.

Reassessing A, B, C

A — Gate calls search. Solves a UX wish by adding a category error and a hidden dependency. It risks applying or suggesting a spec edit in fixed mode and couples the proof gate to the parser/vocabulary. Reject.

B — Keep separate. Correct as an interim state: the gate is safe and the search is diagnostic. But it leaves the overloaded "repair" vocabulary, `Patch.SpecEdit`, the runtime hash check, and the string suggestion in place. It treats the symptom as the architecture.

C — Edge advisor/facade. A good adapter if a combined UX is required. It keeps the core gate clean and makes advice explicit. But if `Patch.SpecEdit` and the three meanings of "repair" remain, C is a layer over an unstable model. It is a migration step, not the destination.

Option D — Typed proposals, advice, and a frozen-spec capability

The architectural alternative is to separate the artifact changes from the diagnostic advice, and to make fixed-spec mode a capability rather than a flag.

A sketch, not an implementation:

```
Artifact      = Specification | Implementation
Proposal      = SpecificationChange of SpecificationProposal
              | ImplementationChange of ImplementationPatch
Advice        = { Target: Artifact; Span: Span; Text: string; Evidence: ... }
FrozenSpec    = a linear capability naming a content-addressed specification
SpecChange    = a capability available only in exploratory mode
Verdict       = Accepted | Rejected of reason
```

Consequences:

- `ImplementationPatch` contains only an implementation body and the identity of the frozen specification. It cannot carry a spec edit.
- A `SpecificationChange` is constructible only with the exploratory `SpecChange` capability. In fixed-spec mode the type does not exist.
- `Refiner` produces a `SpecificationProposal`, not a string in a patch.
- `RepairSearch` produces `Advice`, not a proposal.
- `Acceptance.accept` consumes a `FrozenSpec` and an `ImplementationProposal` and returns a `Verdict`. It never calls the sentence search.
- A combined UX is an edge coordinator over `Advice` and `Verdict`; the core does not know about the coordinator.
- The fixed-spec invariant is structural, not a runtime `SpecEdit` check and a hash comparison.

This fits the project's existing direction: append-only image, pure functions, linear ownership, and named refusal values. It also makes the word "repair" optional rather than load-bearing.

Research notes: how other systems separate these concerns

The following are conceptual references, not claims about Scale's code:

- **Compiler diagnostics versus quick fixes.** A diagnostic reports a problem; a code action or fix-it suggests an edit. Applying the edit is a separate operation. A compiler does not accept its own quick fix as a proof.
- **Machine-applicable suggestions.** `rustc` and `clang-tidy` distinguish a suggestion from the compiler's accept/reject decision. The suggestion has a span and applicability; the decision has evidence.
- **Program repair.** Syntactic repair (edit distance) and semantic repair (test or proof obligation) have different guarantees. Scale currently has both but uses one word.
- **Typed holes and elaborator suggestions.** A hole is a typed obligation with candidate terms; a candidate is not an accepted term.
- **Bidirectional transformations and lenses.** A specification and an implementation are related but not the same artifact; a change in one direction is not automatically a change in the other.
- **"Parse, don't validate" and "make illegal states unrepresentable."** An optional `SpecEdit` is the anti-pattern; a sum type or a capability is the correction.
- **Linear capabilities.** A fixed specification is a linear resource. A spec change requires a capability that fixed mode does not grant; this is the same discipline the project already uses for ownership and actor system messages.
- **Content-addressed artifacts.** If the spec is content-addressed, a patch can name the spec it was built against, and the "same spec" check becomes part of the value's identity rather than a string comparison.

No external source is required to see the pattern: the project already has the right primitives in its own architecture.

Sources consulted: the Language Server Protocol specification separates a `Diagnostic` from a `CodeAction`/quick fix (LSP 3.17); the LSP issue that proposed putting quick fixes directly on diagnostics records the counterargument that fixes should come through the code-action request (LSP issue 581); GHC documents typed holes as obligations with "valid hole fits", not accepted terms (GHC typed holes); and the Hazel work on typed holes makes the same distinction between a hole, its typing context, and a completion (Statically Contextualizing LLMs with Typed Holes). None of these systems equates a suggestion with acceptance.

Fit with Scale's unique architecture

- **Append-only image.** A proposal is a new image value, not a mutation. A `Proposal` record with target, body, span, and evidence fits the image model better than an optional field on a patch.
- **Pure functions.** `Acceptance.accept` can remain a pure function over a frozen spec, a proposal, and a corpus. Advice generation is another pure function.
- **Linear ownership.** A frozen spec is a linear token. Exploratory mode has a spec-change capability; fixed mode does not. This turns a runtime rule into a type-level one.
- **Actors and system messages.** The same separation already appears in D75: runtime-controlled messages are handled before user delivery, and the user cannot receive them. Advice should similarly be handled before acceptance, not mixed into it.
- **MOP/image orientation.** The proposal and advice values can be image-resident or host-level, but the **decision** remains a pure gate. There is no need to make the gate a metaobject.

Open research questions

1. Is `Patch.SpecEdit` ever constructed with `Some` outside tests? The current tree says no. If so, it is a guard for a generator contract, not a feature.
2. Is `Refiner` already the only production spec-change path? If yes, `SpecEdit` is redundant and should be removed rather than renamed.

3. Does any caller actually need one combined result (verdict + suggestion), or are `--diagnose` and `--fixed-spec` intentionally separate?
4. Should a diagnosis carry a structured `Advice` at all, or is a rendered string sufficient at the edge?
5. Is a content-addressed `FrozenSpecification` worth the change, or is the runtime hash check acceptable for now?
6. Does the append-only image already give a natural `Proposal` encoding, or would that be a second representation with no consumer?
7. What is the migration cost of renaming and splitting `Repair`? Which tests encode the current contract rather than the intended one?
8. Which of the three "repair" operations is the product actually trying to optimize: writing the spec, changing the spec, or accepting the implementation?

Recommended next steps

1. Treat A as rejected and B as the safe interim state.
2. Confirm whether `SpecEdit` has a production constructor; if not, remove the field and let the type express the fixed-spec rule.
3. Prototype D in a small module, not in `Repair`: `Proposal`, `Advice`, `FrozenSpec`, `SpecChange` capability, `Acceptance.accept`.
4. Keep the current gate unchanged until the prototype proves the type-level invariant and the migration is understood.
5. Only add the edge coordinator if a combined UX is actually required.

Conclusion

The question "should the fixed-spec gate call the sentence search?" assumes that the two operations share a substrate. They do not.

The likely earlier mistake is the mixed `Patch` type and the single word "repair" covering sentence correction, specification refinement, and implementation acceptance. The gate is not missing a call; the model is missing a distinction.

Fix the distinction first. Then A, B, and C become small, reversible choices rather than architectural decisions.

Implemented API surface (D79)

`FixedSpecificationAcceptance` (`fixed-specification-acceptance.fs`):

- `Mode` (`Exploratory`, `FixedSpecification`), `SourceLocation`, `Witness<'T>`, `CorpusEntry<'T>`, `AcceptanceBundle<'T>`.
- `Target` and `FrozenSpecification`; `freeze`; `specificationHash`.
- `ImplementationProposal` and `propose`; the type has no specification edit.
- `SpecificationProposal`, private `SpecChangeCapability`, `specChangeCapability`, `proposeSpecification`, and `appendSpecification`.
- `accept`, which consumes a frozen specification and an implementation proposal.
- `Verdict`, `makeBundle`, `bundleJson`, `writeBundle`, `corpusJson`, `writeCorpus`, `readCorpus`, `differentialResult`, `conditionWitness`.

`NearestSentence` (`nearest-sentence.fs`):

- `Candidate`; `candidates`, `candidatesWith`, `nearest`, `nearestWith`; `advice` produces `Violations.Advice`.

Diagnostics and presentation:

- `Violation.Suggestion: Advice option` replaces the old string field.
- `Presentation.withNearestSentence` fills advice and `Presentation.render` prints `Suggestion:`.
- `Violations.parseText` and `Span.Start`/`End` supply source character offsets.

Test map:

- Examples: `development/scale/tests.fs` pins nearest-sentence candidates, the layer hook, advice fields, proposal acceptance, numeric targets, corpus regression, falsification, ill-sorted bodies, a proposal against another specification, unencodable targets, differential disagreement, capability refusal in fixed mode, exploratory append, and the frozen specification hash.
- Properties: `properties.fs` pins nearest-sentence idempotence and that an implementation proposal keeps its frozen identity.
- Granular integration: `development/scale/granular-tests.fs` pins the property count, the exploratory capability, a changed-document refusal, the fixed-spec run, and the refinement loop.
- Snapshots: the bundle JSON remains pinned by the granular snapshot tests.
- NativeAOT: `just aot-test` runs the unfiltered example suite.

Boundary. The capability is a private-constructor value, not a type-theoretic linear type. There is no combined verdict-plus-advice facade; `--diagnose` and `--fixed-spec` remain separate. Historical records keep their old names.